Object Oriented Programming (CSEG1044)
Unit 02 – Lecture 05 Notes
Abstract Classes, ‘final‘, and Interfaces (Design Choices)

April 1, 2026

Contents

1 Lecture Snapshot **2**

2 Lecture Overview **2**

3 Prerequisite Bridge **2**

4 Learning Outcomes **2**

5 Suggested Reading Order **3**

6 Key Terms (Quick) **3**

7 Detailed Notes **3**
- 7.1 Abstract classes and methods . . . 3
- 7.2 `final` keyword (design intent) . . . 4
- 7.3 Interfaces: built-in + user-defined . . . 4

8 Running Project Connection **4**

9 Common Mistakes and Confusions **5**

10 Worked Example: `Comparable` for sorting **5**

11 Demo / Observation Task **6**

12 Practice Check (with brief solutions) **6**

13 Self-Study Practice (no solutions) **6**

14 Further Study **6**

15 Exit Questions (with sample answers) **7**

1 Lecture Snapshot

- **Unit focus:** Classes, Inheritance, Packages, and Interfaces
- **Main new ideas:** Abstract classes and methods, `final` keyword (design intent), Interfaces: built-in + user-defined
- **Course thread:** Use Abstract Classes, 'final', and Interfaces (Design Choices) to strengthen the domain model, APIs, and access rules inside Campus Resource Manager.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

2 Lecture Overview

By now you have seen basic classes, inheritance, and an interface intro. This lecture answers the practical design questions:

- When should I use an **abstract class**?
- When should I use an **interface**?
- When should something be **final**?

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

3 Prerequisite Bridge

Before reading the new material in depth, do a fast recall of the older ideas listed below.

- **Inheritance**: Abstract classes extend inheritance by allowing an incomplete but shared parent design. Main reference: `Unit 02 / Lecture 04 slides/notes`.
- **Interface idea**: You already saw interfaces as contracts; this lecture compares them with abstract classes in more depth. Main reference: `Unit 02 / Lecture 03 slides/notes`.

This lecture only revisits those ideas briefly so that the main explanation can stay focused on the new concept sequence.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

4 Learning Outcomes

After this lecture, you should be able to:

- Use abstract classes for shared code + required methods.
- Explain `final` for variables/methods/classes.
- Use interfaces (including `Comparable`, `Runnable`) as contracts.

5 Suggested Reading Order

1. Read the **Prerequisite Bridge** first and confirm each recalled idea.
2. Study the central explanation in this order: *Abstract classes and methods* → ***`final`*** *keyword (design intent)* → *Interfaces: built-in + user-defined.*
3. Trace the worked example line by line before checking short answers.
4. Attempt the practice section without looking at the solution immediately.
5. Finish with the exit questions and further-study suggestions to confirm retention.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

6 Key Terms (Quick)

- **Abstract class**: class that cannot be instantiated; may include abstract methods.
- **Abstract method**: declared without body; must be implemented by concrete subclasses.
- **Interface**: contract specifying methods a class must provide.
- **Final**: "cannot change" (variable), "cannot override" (method), "cannot extend" (class).
- **Contract**: what is promised by a type (what callers can rely on).

7 Detailed Notes

7.1 Abstract classes and methods

Use an abstract class when:

- you want to enforce a common API (method signatures),
- and you also want to share some implementation code across subclasses.

Key properties:

- Cannot instantiate: `new Shape()` is illegal if `Shape` is abstract.
- Subclasses must implement all abstract methods (unless they are also abstract).
- An abstract class can contain concrete fields and methods.

7.2 `final` keyword (design intent)

Final variable: assigned once.

- For primitives: value cannot change.
- For references: reference cannot change, but the object may still change unless the object is immutable.

Final method: cannot be overridden.

- Used to lock down behavior and prevent subclasses from changing critical logic.

Final class: cannot be extended.

- Example: `String` is final (one reason is security/immutability design).

7.3 Interfaces: built-in + user-defined

Use an interface when:

- you want multiple unrelated classes to share a capability (e.g., `Comparable`),
- you want multiple implementations (e.g., different payment methods),
- you want flexible designs with low coupling.

Built-in examples:

- `Comparable<T>`: defines natural ordering (`compareTo`). Used by sorting.
- `Runnable`: defines a task that can run in a thread (`run`).

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

8 Running Project Connection

Use Abstract Classes, 'final', and Interfaces (Design Choices) to strengthen the domain model, APIs, and access rules inside Campus Resource Manager.

- Use Campus Resource Manager as the running case study, or map the same idea to your own project domain if you are using another example.
- Ask where today's idea should live: model, service, DAO, UI, or team workflow.
- Use the lecture example as a smaller version of the same design choice you will make in the capstone.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).;

Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

9 Common Mistakes and Confusions

- Trying to instantiate an abstract class directly.
- Marking too many classes or methods final too early.
- Choosing an interface even when shared state or shared code is central.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

10 Worked Example: `Comparable` for sorting

```java
import java.util.Arrays;

class Student implements Comparable<Student> {
    private final String rollNo;
    private final double cgpa;

    Student(String rollNo, double cgpa) {
        this.rollNo = rollNo;
        this.cgpa = cgpa;
    }

    @Override
    public int compareTo(Student other) {
        // descending order by cgpa
        return Double.compare(other.cgpa, this.cgpa);
    }

    @Override
    public String toString() {
        return rollNo + ":" + cgpa;
    }
}

public class Demo {
    public static void main(String[] args) {
        Student[] a = {
            new Student("21CS1001", 8.2),
            new Student("21CS1002", 9.1),
            new Student("21CS1003", 7.6)
        };
        Arrays.sort(a); // uses compareTo
        System.out.println(Arrays.toString(a));
    }
}
```

11 Demo / Observation Task

Use this block as a short reproducible demo or observation task.

- Reproduce the lecture's main example around *Abstract classes and methods* once without looking at the solution first.
- Change one input, rule, or design choice in Campus Resource Manager and predict the result before checking it.
- Record one success case, one edge case, and one failure case so the idea becomes observable instead of purely theoretical.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

12 Practice Check (with brief solutions)

1. Can we create an object of an abstract class? **Answer:** No.
2. What does `final class` mean? **Answer:** cannot be extended.
3. Why do we use interfaces? **Answer:** contracts + low coupling + multiple implementations.

13 Self-Study Practice (no solutions)

1. Create an abstract class `Shape` with abstract method `area()` and implement `Circle` and `Rectangle`.
2. Create an interface `Payable` and implement it for `Invoice` and `SalarySlip`.
3. Implement `Runnable` in a class `HelloTask` and print a message in `run()` (we will start threads in Unit 03).

14 Further Study

- Revisit the prerequisite references if any recall bullet still feels weak.
- Rework the lecture example with one changed assumption, input, or design choice.
- Read the textbook and documentation references listed in the source line for a second explanation of the same topic.
- Apply the same idea once inside Campus Resource Manager so that the concept moves from theory to design practice.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

15 Exit Questions (with sample answers)

Q1. When do you prefer an interface over an abstract class?

Sample answer: Prefer an interface when you want a contract that many unrelated classes can implement and you want flexibility and low coupling. Use abstract class when you also want to share code and common state.

Q2. What is the meaning of `final` in different contexts?

Sample answer: A final variable cannot be reassigned, a final method cannot be overridden, and a final class cannot be extended.

Q3. When is an interface better than an abstract class?

Sample answer: When you want a loose contract, multiple implementations, and no required shared state.